

Program3

Code a dockerized python flask or node.js application

step 1: create folder

>> mkdir program3

>> cd program3

```
rv24mc053_kavya@kavya-Victus-by-HP-Gaming-Laptop-15-fa1xxx:~$ mkdir program3
rv24mc053_kavya@kavya-Victus-by-HP-Gaming-Laptop-15-fa1xxx:~$ cd program3
rv24mc053_kavya@kavya-Victus-by-HP-Gaming-Laptop-15-fa1xxx:~/program3$ ls
rv24mc053_kavya@kavya-Victus-by-HP-Gaming-Laptop-15-fa1xxx:~/program3$ nano app.py
rv24mc053_kavya@kavya-Victus-by-HP-Gaming-Laptop-15-fa1xxx:~/program3$ nano Dockerfile
rv24mc053_kavya@kavya-Victus-by-HP-Gaming-Laptop-15-fa1xxx:~/program3$ nano requirements.txt
```

step2: create files inside this folder

>> nano app.py

```
rv24mc053_kavya@kavya-Victus-by-HP-Gaming-Laptop-15-fa1xxx: ~/program3
GNU nano 7.2 app.py
from flask import Flask

app = Flask(__name__)

@app.route('/')
def hello():
    return "Hello from docker"

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=8080)
```

>>nano requirements.txt

```
rv24mc053_kavya@kavya-Victus-by-HP-Gaming-Laptop-15-fa1xxx: ~/program3
GNU nano 7.2 requirements.txt
flask==2.3.3
```

>>nano Dockerfile

```
rv24mc053_kavya@kavya-Victus-by-HP-Gaming-Laptop-15-fa1xxx: ~/program3
GNU nano 7.2 Dockerfile
# Use an official Python runtime as a parent image
FROM python:latest

# Set the working directory in the container
WORKDIR /app

# Copy the current directory contents into the container at /app
COPY . .

# Install any needed dependencies from requirements.txt
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Make port 5000 available to the world outside this container
EXPOSE 8080

# Run the application
CMD ["python", "app.py"]
```

step3: build the dockerfile

>> docker build -t program3 .

```
rv24mc053_kavya@kavya-Victus-by-HP-Gaming-Laptop-15-fa1xxx:~/program3$ docker build -t program3 .
[+] Building 270.1s (10/10) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 500B
=> [internal] load metadata for docker.io/library/python:latest
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [1/5] FROM docker.io/library/python:latest@sha256:1ad1a43b5e2478e62056bbc28028afd858185d73bf4d6a439cbb058b6800a
=> => resolve docker.io/library/python:latest@sha256:1ad1a43b5e2478e62056bbc28028afd858185d73bf4d6a439cbb058b6800a
=> => sha256:1ad1a43b5e2478e62056bbc28028afd858185d73bf4d6a439cbb058b6800a96d 10.95kB / 10.95kB
=> => sha256:c4a87c25190f591b29fb62b4ee424ea9907b4a1997357ed47e82d6db98c80b12 2.32kB / 2.32kB
=> => sha256:0134b02fcae7fd813645ca13d87412024d3c4324460c0b55e84015951284ad58 6.48kB / 6.48kB
=> => sha256:e3143549f2b8b3ad8d79efdc47824641c6771796b3770f3c637a38aabd2b3462 25.62MB / 25.62MB
=> => sha256:13cc39f8244ac66bf1dd9149e1da421ab1bbc80d612dc14fe368753e7be17b33 49.29MB / 49.29MB
=> => sha256:72e8e93b0d018b135d833207c6feaba205653ac52e0a80d214477ec0de239dee 67.78MB / 67.78MB
=> => sha256:23d55a674b2e3641a37b7f2053f00af6fe4f2a89576fffb917d4b7b4deaf24591 235.94MB / 235.94MB
=> => sha256:8f5fb10be9d4525dc6941af4e2a587d1d3edbbb7442cdad0eebb1a3806b183dd 6.09MB / 6.09MB
=> => extracting sha256:13cc39f8244ac66bf1dd9149e1da421ab1bbc80d612dc14fe368753e7be17b33
=> => extracting sha256:e3143549f2b8b3ad8d79efdc47824641c6771796b3770f3c637a38aabd2b3462
=> => sha256:2fbf02c04055ee28543076a502ce51fa41895e386e4462739fb1526dc371425e 29.23MB / 29.23MB
=> => extracting sha256:72e8e93b0d018b135d833207c6feaba205653ac52e0a80d214477ec0de239dee
=> => sha256:149075fcccc1b52406789bff7063813f1c14baf10b165df0e1e00fcb0ca415e5 248B / 248B
=> => extracting sha256:23d55a674b2e3641a37b7f2053f00af6fe4f2a89576fffb917d4b7b4deaf24591
=> => extracting sha256:8f5fb10be9d4525dc6941af4e2a587d1d3edbbb7442cdad0eebb1a3806b183dd
=> => extracting sha256:2fbf02c04055ee28543076a502ce51fa41895e386e4462739fb1526dc371425e
=> => extracting sha256:149075fcccc1b52406789bff7063813f1c14baf10b165df0e1e00fcb0ca415e5
=> [internal] load build context
=> => transferring context: 93B
=> [2/5] WORKDIR /app
=> [3/5] COPY . .
=> [4/5] COPY requirements.txt .
=> [5/5] RUN pip install --no-cache-dir -r requirements.txt
=> exporting to image
=> => exporting layers
=> => writing image sha256:e5515b23216e79a371355b1139b5ca94e5231bdac80561f857261f8c608718a4
=> => naming to docker.io/library/program3
```

Step 4: Run the Dockerfile and expose to the port and open the browser to see the message from the Docker.

>> docker run -p 5000:5000 program3

```
rv24mc053_kavya@kavya-Victus-by-HP-Gaming-Laptop-15-fa1xxx:~/program3$ docker run -p 5001:5000 program3
* Serving Flask app 'app'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://172.18.0.3:5000
Press CTRL+C to quit
```

